

# ComponentGraph vs ComputationGraph

Updated 21 September 2026

Comparison of the current agentofreality-parallel-computation-graph branch, not a published release.

## Bottom line

**ComputationGraph is a working alternative with better control over components and more flexible processing arrangements. It is not yet a faster replacement.**

The main scheduling problem identified in the earlier report has been fixed: independent queries now run in separate, owned tasks. Performance improved, but ComputationGraph still takes **23.1% longer than ComponentGraph** on the measured workload.

Continue developing it, but keep ComponentGraph as the default for now.

## What both engines already support

Both use the same underlying query-evaluation code. Both support the existing source, query and reaction interfaces, Cypher/GQL, joins, middleware, initial data loading, scheduled work and recovery where the source and storage support it. Changing engines does not require a new plugin interface.

Two recent improvements apply to **both engines**, not just ComputationGraph:

- Queued query inputs are ordered by source-reported event time, then source order in the query definition, then source-maintained sequence number. This does not guarantee an order over events that have not arrived.
- Server reports a loaded plugin's actual package version, not its configuration-format version.

## The meaningful differences

| Area                      | ComponentGraph                                                                          | ComputationGraph                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Who manages execution     | The graph records components and status; source, query and reaction managers run them.  | The graph owns components and connections and decides when to create, start, stop or replace them.       |
| Processing arrangements   | Built around source-to-query-to-reaction pipelines, with middleware for preprocessing.  | Also supports custom transformers, sinks and services, including paths with no query.                    |
| Connection checks         | Checks follow the established component interfaces and dependencies.                    | Components declare named inputs/outputs and required formats and delivery behaviour.                     |
| Adding a component        | Addition can wait for initialization; a failed source initialization removes its entry. | An accepted node appears first. Later initialization/startup failures stay visible on it.                |
| Readiness                 | Existing status and start methods.                                                      | Separate handles can wait for creation or actual readiness; accepting an addition is not readiness.      |
| Changing a running system | Component-specific update and removal APIs.                                             | Can preview changes and reconnect affected components. Old handles cannot control replacement instances. |
| Inspection                | Established component status, relationships and events.                                 | Also shows nested query graphs, known resources, plugin references and recent graph changes.             |
| Readiness / cleanup       | Managed through component managers and plugins.                                         | Separate readiness messages and explicit cleanup ownership.                                              |

The standard DrasiLib and Server APIs can select either engine. A DrasiLib instance can also host additional computation graphs alongside its existing ComponentGraph pipeline. Neither arrangement automatically migrates saved state.

## Current performance

The latest controlled comparison used **100,000 inputs and two Building Comfort queries**: room results and floor aggregation. Each engine had one warm-up and five measured runs, with alternating engine order and the same release binary.

| Engine           | Median elapsed time | Range across measured runs |
|------------------|---------------------|----------------------------|
| ComponentGraph   | 3.804 seconds       | 3.781-3.877 seconds        |
| ComputationGraph | 4.681 seconds       | 4.629-4.719 seconds        |

The scheduling fix reduced ComputationGraph's median elapsed time from **7.152 to 4.681 seconds**, a **34.5% reduction**. Queries previously shared one execution task; their separate tasks now allow independent queries to use different CPU workers.

Per-change timing and CPU samples show that converting source events and query results between formats remains a substantial cost. Query timings now include commit. Timing intervals can include waiting and are not pure CPU measurements.

**Every run matched 99,981 room results and 49,860 floor results, including ordered result hashes.** The comparison checks result values and their order; changing timestamps and delivery bookkeeping are excluded.

The measurement window starts at the first observed result and ends when both full result streams finish. It excludes startup/shutdown but includes generation, queues, result collection and hashing. Profiling-file output was disabled during timing runs.

This was an in-memory, in-process workload, without Server, network transports or dynamically loaded plugins. It is evidence for this workload, not proof of equal behaviour or performance in every deployment.

## Recovery and operational behaviour

ComputationGraph now includes the delivery, checkpoint and persistent-storage fixes brought over from main. Query state and outputs can commit together with a supporting storage provider. Reactions save completed handling, not just queue acceptance. Recovery can replay available missed outputs, and old checkpoints cannot silently apply to a recreated query.

Dynamically loaded sources can replay when their plugin supports it. The earlier report's blanket restriction is outdated.

Transient sources cannot resend lost data. Persistent indexes alone do not make query output durable. Neither engine guarantees one-time external actions.

Tests cover both modes, real components, pause/restart/shutdown, persistent recovery and Server integration. They do not prove equivalence in every deployment.

## Main remaining work

- **Reduce the remaining performance gap** and measure more queries, multiple sources, persistent storage and real plugins.
- **Create source/reaction plugins from configuration inside the standard add APIs.** They currently receive preconstructed objects, so constructor failures before the call cannot appear as failed graph nodes.
- **Record all provider relationships and creation details.** Graph export does not yet capture everything needed to recreate every plugin, storage provider and connection to a supplied object.

- **Move more deployment/cloning work into reusable library APIs** and expose the richer graph controls through Server.
- **Provide an explicit migration path** before changing the default engine. Selecting ComputationGraph does not transfer ComponentGraph's stored state.
- **Decide whether all-or-nothing batch deployment is needed.** Additions deliberately remain visible, including failures. Saving one query's storage updates together does not make a whole deployment all-or-nothing.

## Versions and supporting material

The assessment uses these committed revisions: drasi-core a7892b00, drasi-server 58669661, test-infra d3fbbbb4.

The measured runtime fix is core commit 6a6301bc; the later commits listed above update documentation, not runtime behaviour. The benchmark evidence is retained in this workspace session's `parallel-performance-summary.json`, `ranked-baseline/` and `parallel-query-benchmark/` records.

For details, use the [design overview](#), [usage guide](#) and [configuration reference](#).